Code walkthrough

program_counter.v

Program Counter (PC)

What it does	Holds the address of the instruction being fetched
Where it works	IF stage
Size	30 lines, explained in 3 small parts
Modules in this file	program_counter
Calls (uses)	nothing (no other modules)
Called by	cpu_top.v - instance PC

The PC is a 4-bit register. Its value is the address sent to the instruction memory. Every clock it moves to the next address (or to a jump address).

Map of the file

Part	Lines	What this part is
1	1-15	Header comment
2	16-22	Ports
3	23-30	Clocked behaviour

> How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

Part 1 · Header comment (lines 1-15)

program_counter.v lines 1-15

```
 1  // ======================================================================
 2  // FILE: program_counter.v
 3  // The PC holds the address of the instruction being FETCHED (0 to 15).
 4  //
 5  // Interface (as in the project plan):
 6  //   clk, rst, pc_en, pc_next, pc_out
 7  //
 8  // In the 3-stage pipeline a new instruction is fetched every clock,
 9  // so pc_en is kept at 1 by cpu_top. pc_next is PC + 1, or the jump
10  // address when a jump is taken in the EX stage.
11  //
12  // CONNECTIONS
13  //   Calls       : nothing (no other modules)
14  //   Called by   : cpu_top.v - instance PC
15  // ======================================================================
```

- Lists the interface from the project plan (clk, rst, pc_en, pc_next, pc_out) and explains that in the pipeline the PC changes every clock.
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

Part 2 · Ports (lines 16-22)

program_counter.v lines 16-22

```
16  module program_counter (
17      input            clk,
18      input            rst,
19      input            pc_en,       // 1 = load pc_next
20      input      [3:0] pc_next,     // the next address
21      output reg [3:0] pc_out       // current fetch address
22  );
```

- `pc_en`: when 1 the PC may change. cpu_top ties it to 1 so a new instruction is fetched every clock.
- `pc_next`: the address to load, coming from the PC MUX (PC + 1 or jump address).
- `pc_out` is declared `reg` because it is given a value inside an `always` block.

Part 3 · Clocked behaviour (lines 23-30)

program_counter.v lines 23-30

```
23      always @(posedge clk) begin
24          if (rst)
25              pc_out <= 4'd0;       // after reset, start at address 0
26          else if (pc_en)
27              pc_out <= pc_next;    // go to the next address
28          // if pc_en = 0, pc_out keeps its old value
29      end
30  endmodule
```

- `always @(posedge clk)` = do this at every rising edge of the clock. This makes a real register (flip-flops).
- Reset has priority: `pc_out <= 0`, so the program always starts at address 0.
- Otherwise, if pc_en is 1, the PC takes the new address. If pc_en were 0 it would keep its old value.
- <= is the non-blocking assignment, always used for clocked registers.

> **In short:** Reset → 0; every clock → next address. **Example:** PC = 0101 and pc_next = 0110. At the rising edge pc_out becomes 0110. If rst = 1 at that edge, pc_out becomes 0000 instead.